arXiv:2406.02128v2 [cs.LG] 28 Oct 2024

Iteration Head: A Mechanistic Study of Chain-of-Thought

Vivien Cabannes FAIR, Meta AI
Charles Arnal Datashape, INRIA
Wassim Bouaziz FAIR, Meta AI
Alice Yang FAIR, Meta AI
Francois Charton FAIR, Meta AI
Julia Kempe Courant University and Center for Data Science, NYU & FAIR, Meta AI

Abstract

Chain-of-Thought (CoT) reasoning is known to improve Large Language Models both empirically and in terms of theoretical approximation power. However, our understanding of the inner workings and conditions of apparition of CoT capabilities remains limited. This paper helps fill this gap by demonstrating how CoT reasoning emerges in transformers in a controlled and interpretable setting. In particular, we observe the appearance of a specialized attention mechanism dedicated to iterative reasoning, which we coined "iteration heads". We track both the emergence and the precise working of these iteration heads down to the attention level, and measure the transferability of the CoT skills to which they give rise between tasks.

1 Introduction

In the rapidly evolving field of artificial intelligence, Large Language Models (LLMs) have emerged as a pivotal component [45]. Their ability to understand, generate, and manipulate human language has opened up new avenues towards advanced machine intelligence. Interestingly, despite being primarily trained on next-token prediction tasks, LLMs are able to produce much more sophisticated answers when asked to generate steps of reasoning [30, 58]. This phenomenon, often referred to as Chain-of-Thought (CoT) reasoning, and illustrated on Table 1, appears paradoxical: on the one hand, LLMs are not explicitly programmed to reason; on the other hand, they are capable of following logical chains of thoughts to produce relatively complex answers.

Table 1: Chain-of-Thought consists in eliciting reasoning steps before answering (A) a question (Q).

[Q] What is $8 \times 8 \times 3$?	:	[A] 210.
[Q] What is $8 \times 8 \times 3$? Take it step by step.	:	[A] $8 \times 8 = 64$, $64 \times 3 = 192$. It is 192.

Recent studies have shown that the class of problems a transformer can solve with single-token prediction, i.e. by outputting a single token meant to be the correct answer, is rather limited [24, 54, 15]. In contrast, when transformers are allowed to freely generate tokens before providing a final answer, they can use those generated tokens as a tape to emulate a form of Turing machine [48]. This enables them to solve a larger class of problems [38, 18, 39, 35]. However, our understanding of why and how transformers gain CoT abilities when trained with next-token predictions remains limited. We aim to provide insights on the matter.

Summary of Contributions. We adopt a “mechanistic interpretability” approach [see 17]: we work with simple, controlled problems and architectures that capture the key aspects of the problem and allow us to observe and analyze, down to the network’s weights and attention, the emergence of CoT in our models. In practice:

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

- We describe the simple yet rich setting of iterative tasks and iterative algorithms, including three simple examples: a copying, a polynomial iteration, and the parity problems.
- We explain why such problems are hard to solve for transformers with single-token prediction. Conversely, we describe how a certain distribution of weights within the first two attention layers of a transformer, which we call an "iteration head", enables a transformer to solve iterative tasks with CoT reasoning with relative ease.
- We hypothesize that iteration heads naturally appear in transformers trained on (hard enough) iterative tasks, and verify this hypothesis in small-scale experiments.
- Ablation studies demonstrate the impact of the training set and choice of hyperparameters in their emergence. We also observe the good transferability of the iterative reasoning skills granted by the attention heads from one iterative task to another, from which we deduce the usefulness of data curation.

Our controlled yet illustrative experimental setup sheds light on the emergence of CoT capabilities in larger LLMs, whose attention patterns are much harder to interpret. In particular, our experiments suggest that transformers are likely to develop "inner circuits" specially dedicated to multistep reasoning, which can then be applied, in combination with more specialized skills, to a variety of tasks that share the same underlying logical structure. This gives a credible explanation of the strong CoT reasoning capabilities of current state-of-the-art LLMs, as their training corpora (human-written texts, computer code) include many examples of complex multistep reasoning.

Related Work. This work is set in the realm of mechanistic interpretability [e.g., 43, 7]. A top-down line of work is trying to explicit algorithms implemented by transformers in the wild [e.g. 57, 20, 25], although some findings might be fallacious [9]. A bottom-up line of work, to which we belong, consists in building understandings from small models that are relevant for bigger models, in particular regarding in-context learning [see, e.g. 61, 19, 8, 23, 2, 34, 49, 16, 60]. In-context learning relates to the reproduction of reasoning patterns that appear in a prompt or context [10]. In contrast, our study of CoT relates to reproducing reasoning patterns that appear in the training set.

2 Controlled Setup: Learning Iterative Algorithms

Human language and human reasoning are often organized in a multistep, cumulative fashion, with each new thought or group of sentences building upon the ones that precede to work towards some final conclusion. LLMs naturally benefit from learning such reasoning patterns: not only are they prevalent through much of their training data, but they also represent an efficient way to divide the total processing effort required into easier intermediate steps. In what follows, *we choose to focus on iterative algorithms and iterative tasks as a controlled proxy for more general forms of CoT reasoning*. Indeed, though conceptually simple, iterative algorithms exhibit a key property: they are simultaneously hard to learn for transformers using next-token predictions, and comparatively easy to learn using CoT reasoning. As such, iterative tasks are ideally suited to illustrate the usefulness of CoT reasoning, and to study its emergence.

Algorithm 1 Iterative Schemes

$s = \texttt{Init}$
for x in $\texttt{Sequence}$ **do**
  $s \leftarrow F(s, x)$;
end for
return s

We define *iterative algorithms*, or iterative schemes, as follows: an iterative algorithm is the combination of an input sequence, denoted as $\texttt{Sequence}$, and made of L elements $(x_t)_{t\in[L]}$ (with $[L] = \{1, \cdots, L\}$), and an internal state, denoted as s, initialized to some default value $s_0 = \texttt{Init}$, and updated as the sequence is processed according to some rule $s_t = F(s_{t-1}, x_t)$ for some function F. Pseudo-code illustrating the concept is provided by Algorithm 1, see also Figure 1. By extension, we informally call iterative task a task which is naturally solved by outputting the end product of some iterative algorithm applied to some input sequence. As an example, consider the *parity problem*, i.e. the problem of computing the parity of the sum of a sequence of 0s and 1s: it can be easily framed as an iterative task. Using the notations of Algorithm 1, let the initial state $\texttt{Init}$ be equal to 0, and let $F(s, x)$ be equal to 0 if s is equal to x, and 1 otherwise. Then the final s_L gives the parity of the sum. Although this task could also be solved in a non-iterative fashion, the iterative solution can be seen as simpler and more parsimonious.

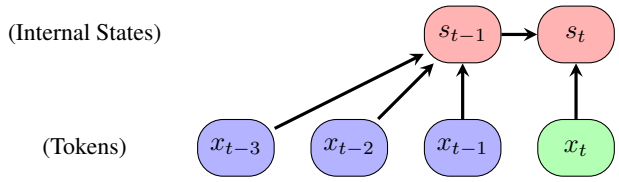

Figure 1: Arguably, reasoning involves updating an internal state (red) as new information is processed (green). The diagram above, where each element represents a piece of information, is an abstract depiction of this idea. This observation motivates our use of iterative tasks as a proxy for more general reasoning processes. At first glance, a limitation of transformers is their lack of an internal state, which makes it challenging to implement this diagram [32].

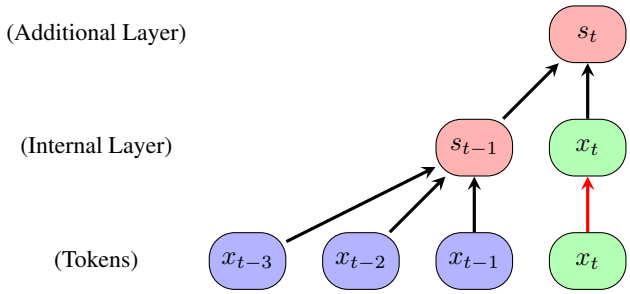

Figure 2: A single transformer layer cannot implement the diagram from 1, as it cannot access its previous outputs. This limitation can be bypassed by stacking transformer layers, as illustrated here. The red arrow indicates a residual connection. This naive method requires as many layers as there are reasoning hops.

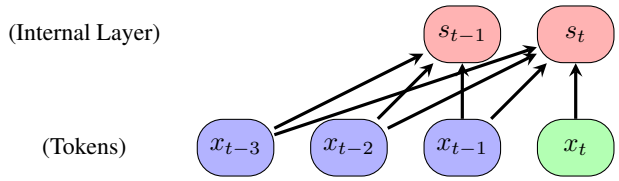

Figure 3: Alternatively, a transformer could compute each state s_t from scratch. This implementation does not require additional layers, but it is not parsimonious, which could lead to computational inefficiencies. This explains the difficulty for a transformer to output the final answer of a chain of reasoning within a token (i.e., with next-token prediction, and without chain-of-thought)

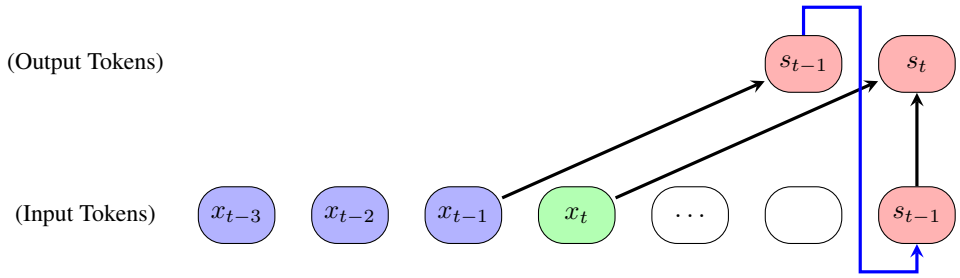

Figure 4: Chain-of-thought addresses this issue by explicitly representing the reasoning process in token space. The auto-regressive nature of LLMs (blue arrow) allows for the implementation of iterative algorithms, as long as the states are encoded in token space. A concrete implementation of such a mechanism, which we call *iteration head*, is described in Section 3. In practical applications of LLMs, one could imagine earlier layers summarizing the t-th input sentence (or some other coherent semantic information of varying token length) into x_t, as well as summarizing the generated CoT sentences into some s_{t-1}, with later layers translating the state s_t into readable text.